

Software Requirements Specification (SRS)

Campus Event Coordination System (CECS)

SWE 2300 (Spring 2026)

Team

Project Leader: Timilsena, Kamal Raj (hdd5ps)

Requirements/Design Lead: Bryington, Mason Toby (xur6rh)

Implementation Lead: Muwanguzi, Precious Solomon (max7jn)

April 2026

Contents

1	Introduction & Scope	2
1.1	Purpose	2
1.2	Problem Statement	2
1.3	Business Objectives	2
1.4	System Boundaries	2
1.4.1	Included (In Scope)	2
1.4.2	Excluded (Out of Scope)	2
2	Stakeholders	3
3	Functional Requirements	3
3.1	Core Requirements (FR-01 to FR-18)	3
3.2	New Features (FR-19 to FR-23)	4
4	Nonfunctional Requirements	4
4.1	Original Nonfunctional Requirements (NFR-01 to NFR-09)	4
4.2	New Nonfunctional Requirements (NFR-10 to NFR-12)	5
5	Requirement Conflicts & Resolutions	5
6	Requirements Modeling	5
6.1	Scenario-Based Model (Use Cases)	5
6.1.1	UC-01: Browse & Filter Events	5
6.1.2	UC-02: Create Event (Draft → Submit for Approval)	6
6.1.3	UC-03: Approve or Reject Event	6
6.2	Data Model (Entity-Relationship Diagram)	6
6.3	Class Model	7
6.4	Behavioral Model (State Diagram for Event)	7
6.5	Model Diagrams (4 Images)	8
7	Traceability Matrix	10
8	Validation & Reflection	10
8.1	Validation Summary	10
8.2	Reflection Questions	10
8.3	Known Issue / Future Enhancement	11

1 Introduction & Scope

1.1 Purpose

This document specifies the functional and nonfunctional requirements for the Campus Event Coordination System (CECS). It is intended to guide implementation, testing, and validation of the system.

1.2 Problem Statement

Campus events are advertised across fragmented channels (emails, posters, social media, informal messages), causing low visibility, missed opportunities, and scheduling/resource conflicts. Cultural-credit check-in can be confusing or unreliable, and students lack a unified way to locate event locations quickly.

1.3 Business Objectives

- Centralize event discovery to increase student awareness and participation.
- Improve reliability and traceability of cultural-credit eligibility and attendance tracking.
- Reduce scheduling/resource conflicts using structured event lifecycle and approvals.
- Support navigation using a campus map pin and an external directions link.

1.4 System Boundaries

1.4.1 Included (In Scope)

- Authentication and role assignment (Student / Staff / Administrator).
- UVA-Wise institutional integration: enforce @uvawise.edu domain validation on login and signup. (New)
- Event browsing (chronological feed), search and filtering by tags/time/location/cost/cultural credit eligibility.
- Event creation/editing by staff with Draft → Pending → Approved/Rejected lifecycle.
- Administrator approval workflow for events and role requests. (Updated)
- Cultural credit status visibility and attendance check-in with timestamp and status.
- Multi-method check-in: Near Field Communication (NFC), Quick Response (QR), and PIN. (New)
- Map v1: embedded pin + external directions link only (no in-app routing).
- Save filtering preferences and reminders (app-level).
- Account Settings page (profile updates, current role, request history). (New)

1.4.2 Excluded (Out of Scope)

- Payment/ticket transactions.
- Full in-app GPS routing / turn-by-turn navigation.

- Enterprise analytics dashboards.
- External calendar integrations in the prototype.

2 Stakeholders

- **Student (Primary)**: discover events, view cultural credit eligibility, check in reliably, find locations quickly.
- **Staff/Club Officer (Primary)**: create/edit events, publish accurate info, manage drafts, tag events correctly.
- **Administrator (Primary/Organizational)**: approve/reject events, review role requests, enforce policy.
- **Student Affairs (Organizational)**: audit cultural credit fulfillment; needs accurate timestamps and traceability.
- **Facilities Staff (Organizational)**: prevent room/resource conflicts; needs schedule clarity.
- **University IT (Technical)**: secure authentication/integration; data integrity and reliability.
- **Campus Security (Organizational)**: visibility into large events and locations.

3 Functional Requirements

3.1 Core Requirements (FR-01 to FR-18)

- **FR-01**: The system shall authenticate users using institutional login for the prototype (mock credentials acceptable).
- **FR-02**: The system shall assign exactly one role to each authenticated user (Student, Staff, Administrator).
- **FR-03**: The system shall display a chronological feed of approved upcoming events.
- **FR-04**: The system shall allow users to view an event detail page including title, host, date/time, location, cost, description, tags, and cultural credit status.
- **FR-05**: The system shall allow users to filter events by tags (e.g., “Free Food”, “Free Entry”).
- **FR-06**: The system shall allow users to filter events by time and location.
- **FR-07**: The system shall allow users to filter events by cost (Free vs Any).
- **FR-08**: The system shall allow users to filter events by cultural credit eligibility (Eligible vs Any).
- **FR-09**: The system shall allow staff users to create events including title, date/time, location, description, cost, and cultural credit designation.
- **FR-10**: The system shall allow staff users to save events as Draft and submit events for approval (status becomes Pending Approval).
- **FR-11**: The system shall allow administrators to approve or reject pending events.

- **FR-12:** The system shall prevent events that are not approved from appearing in the public event feed.
- **FR-13:** The system shall record a timestamped attendance check-in for a student at an event and mark status as “Checked in”.
- **FR-14:** The system shall provide a fallback check-in option when primary check-in fails or is unsupported.
- **FR-15:** The system shall display event location using an embedded mapping interface with a pin at the event location.
- **FR-16:** The system shall provide a link to open directions in an external mapping application.
- **FR-17:** The system shall allow users to save event filtering preferences and apply them on future sessions.
- **FR-18:** The system shall allow users to enable event reminders for saved/upcoming events.

3.2 New Features (FR-19 to FR-23)

- **FR-19:** The system shall enforce strict @uvawise.edu email domain validation on both login and signup.
- **FR-20:** The system shall allow users to check in to events using Near Field Communication (NFC), Quick Response (QR) code scanning, or a PIN code via a multi-method check-in user interface.
- **FR-21:** The system shall allow users to request elevated privileges (Staff or Administrator) by submitting a justification of at least **20 characters**.
- **FR-22:** The system shall provide an Administrator dashboard to view, approve, or reject pending role requests and include reviewer notes.
- **FR-23:** The system shall provide an Account Settings page allowing users to update profile information (name, email), view current role, and view request history/status.

4 Nonfunctional Requirements

4.1 Original Nonfunctional Requirements (NFR-01 to NFR-09)

- **NFR-01 (Performance):** The event feed shall load within 2 seconds under 500 concurrent users.
- **NFR-02 (Performance):** Filtering operations shall return updated results within 1 second.
- **NFR-03 (Security):** User credentials shall be stored using salted cryptographic hashing.
- **NFR-04 (Security):** Administrative actions shall require Role-Based Access Control (RBAC) checks on every request.
- **NFR-05 (Reliability):** Approved event and attendance data shall persist after a system restart.
- **NFR-06 (Availability):** The system shall maintain 99% availability during standard academic hours.

- **NFR-07 (Usability):** A first-time user shall navigate from login to viewing an event detail page within ≤ 3 interactions.
- **NFR-08 (Maintainability):** The system shall separate business logic from presentation logic to support future extensibility.
- **NFR-09 (Portability):** The application shall render correctly on modern mobile browsers (iOS Safari and Android Chrome).

4.2 New Nonfunctional Requirements (NFR-10 to NFR-12)

- **NFR-10 (Usability/Reliability):** The system shall provide multi-method event check-in (NFC/QR/PIN) such that at least one method is usable for the user's device context.
- **NFR-11 (Security):** The system shall enforce RBAC with an approval workflow for privilege escalation requests (role request submission + administrator approval/rejection + reviewer notes).
- **NFR-12 (Security):** The system shall enforce institutional email domain validation to restrict access to @uvawise.edu accounts.

5 Requirement Conflicts & Resolutions

- **Conflict: Low-friction login vs security strength.**
Resolution: Keep login simple but enforce domain validation and RBAC checks server-side on every privileged action.
- **Conflict: Rich filtering vs performance.**
Resolution: Limit expensive combinations if needed, optimize queries/caching, and track response time.
- **Conflict: Embedded mapping vs scope control (mobile usability).**
Resolution: Map v1 is pin + external directions link only; no full routing application programming interfaces.
- **Conflict: Free-text tags vs accurate student filtering.**
Resolution: Use controlled tags (dropdown/whitelist) for filtering.
- **Conflict: Multi-method check-in vs usability complexity.**
Resolution: One check-in screen that defaults to the best available method and offers the others as explicit fallback options.

6 Requirements Modeling

6.1 Scenario-Based Model (Use Cases)

6.1.1 UC-01: Browse & Filter Events

Primary Actor: Student

Preconditions: User is authenticated.

Main Flow:

1. Student opens the event feed.
2. System displays chronological approved events.

3. Student selects filters (tags/time/location/cost/credit).
4. System displays filtered results.

Alternative Flow: Student clears filters to return to full feed.

Exception Flow: No matches → system shows “No results” and suggests clearing filters.

6.1.2 UC-02: Create Event (Draft → Submit for Approval)

Primary Actor: Staff/Club Officer

Preconditions: User has Staff role.

Main Flow:

1. Staff selects “Create Event”.
2. Staff enters required event fields.
3. Staff submits event.
4. System sets status to Pending Approval.

Alternative Flow: Staff saves as Draft instead of submitting.

Exception Flow: Missing required fields → system blocks submit and shows validation messages.

6.1.3 UC-03: Approve or Reject Event

Primary Actor: Administrator

Preconditions: User has Administrator role; event is Pending Approval.

Main Flow:

1. Administrator views pending events.
2. Administrator selects an event and chooses Approve.
3. System changes status to Approved and makes it visible in the feed.

Alternative Flow: Administrator chooses Reject → status becomes Rejected.

Exception Flow: Event already processed → system prevents duplicate action.

6.2 Data Model (Entity-Relationship Diagram)

Entities (updated):

- **User**(UserID, name, email, role)
- **Event**(EventID, title, datetime, description, cost, creditEligible, status, hostUserID, locationID)
- **Location**(LocationID, name, coordinates)
- **Tag**(TagID, name)
- **EventTag**(EventID, TagID)
- **AttendanceCheckIn**(CheckInID, EventID, UserID, timestamp, status, method {NFC, QR, PIN})
- **Preference**(PreferenceID, UserID, savedFilters, reminderEnabled, reminderTiming)
- **RoleRequest**(RequestID, UserID, requestedRole, justification, status, reviewerID, reviewerNotes, createdAt, decidedAt)

6.3 Class Model

Classes:

- **User** (login(), logout()) with subclasses **Student**, **Staff**, **Administrator**
- **Event** (publish(), setStatus(), edit())
- **EventService** (createEvent(), updateEvent(), searchEvents(), applyFilters())
- **ApprovalService** (approveEvent(), rejectEvent())
- **CheckInService** (checkInNFC(), checkInQR(), checkInPIN(), recordTimestamp(), fallbackCheckIn())
- **MapService** (showPin(), openExternalDirections())
- **RoleRequestService** (submitRequest(), reviewRequest(), listPendingRequests())
- **AccountSettingsService** (updateProfile(), viewRole(), viewRequestHistory())

6.4 Behavioral Model (State Diagram for Event)

States: Draft → Pending Approval → Approved → Archived, plus Rejected and Canceled.
Transitions:

- Draft → PendingApproval on submit()
- PendingApproval → Approved on approve()
- PendingApproval → Rejected on reject()
- Approved → Archived on end_of_event()
- Approved → Canceled on cancel()
- Rejected → Draft on revise()

6.5 Model Diagrams (4 Images)

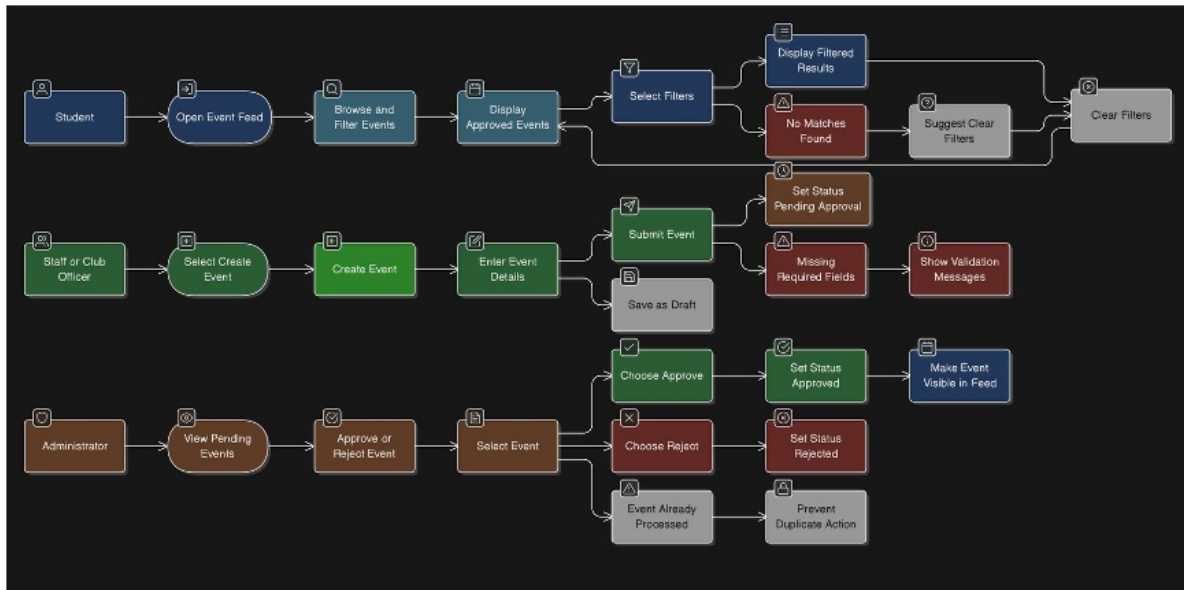


Figure 1: Use Case Diagram

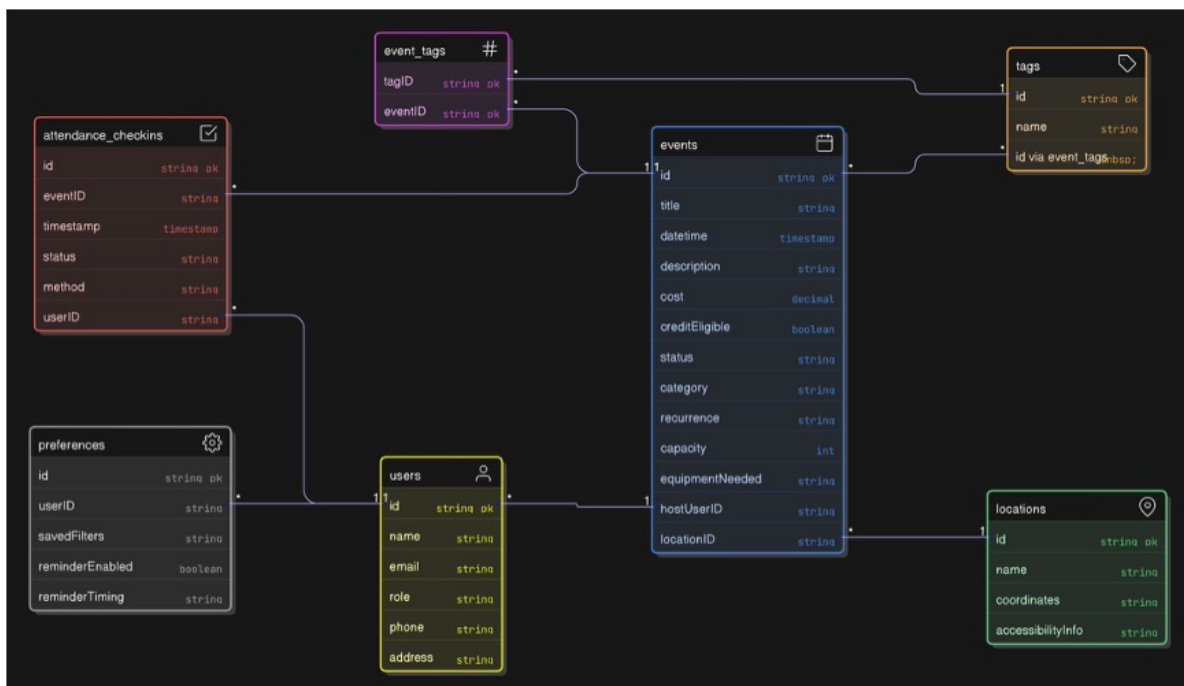


Figure 2: Entity-Relationship Diagram (ERD)

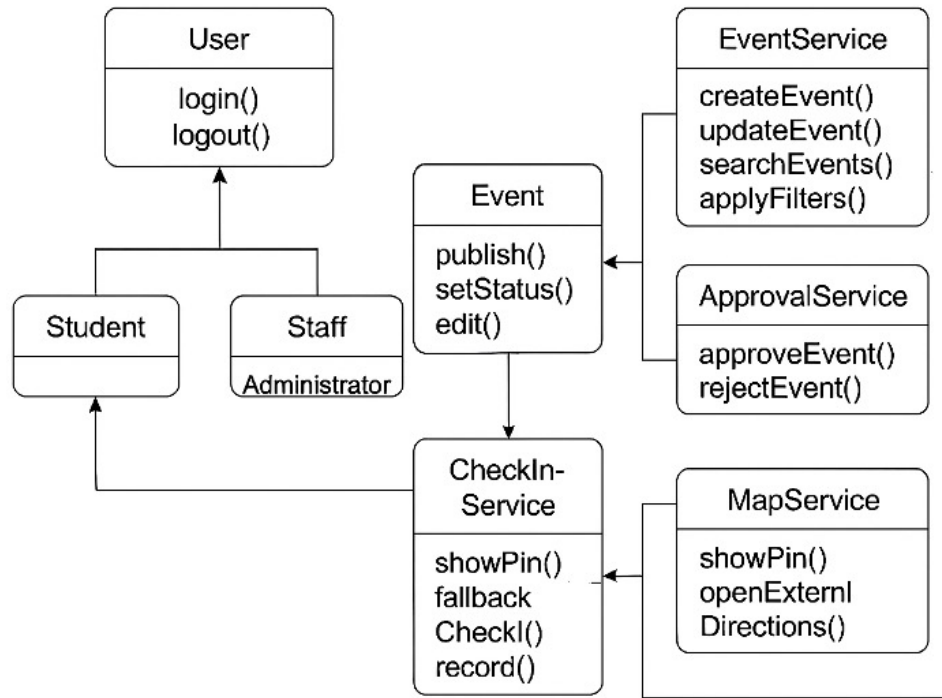


Figure 3: Class Diagram

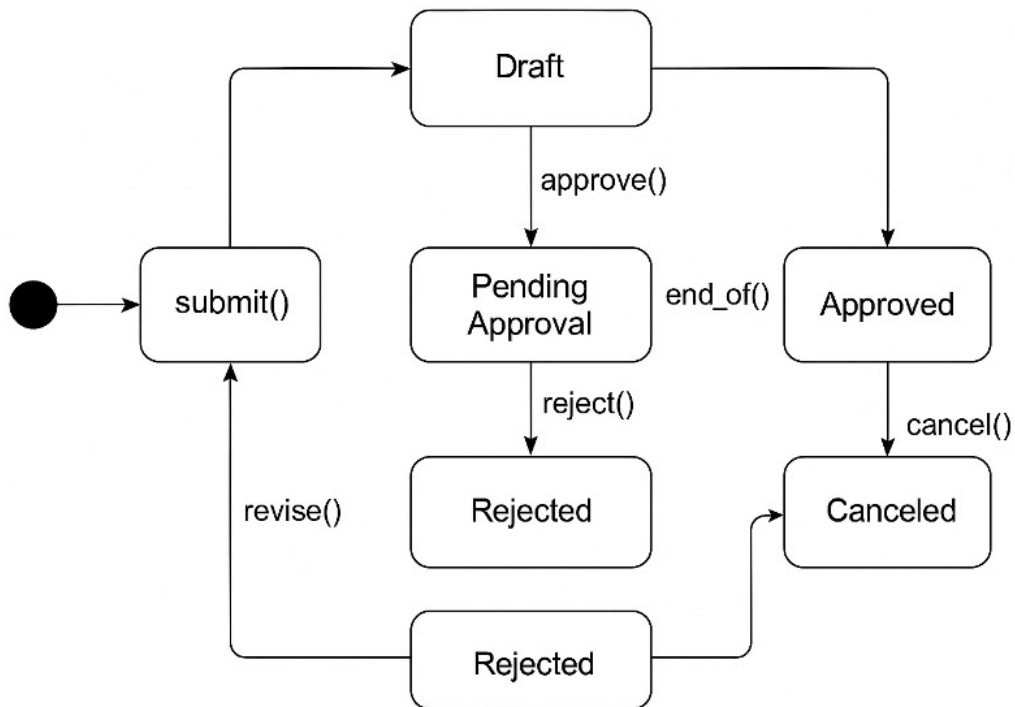


Figure 4: Event State Diagram

7 Traceability Matrix

Req ID	Modeled In	Validated By
FR-01	UC-01 (precondition), Class(User)	Login test + role assignment test
FR-02	Use case model, class model	Role-based access tests
FR-03	UC-01, ERD(Event.status)	Feed shows only Approved events
FR-04	UC-01/Details, ERD(Event)	Event detail page field verification
FR-05	UC-01, ERD(Tag/EventTag)	Filter correctness tests
FR-06	UC-01, ERD(Location)	Time/location filter tests
FR-09	UC-02, Class(EventService)	Create event scenario tests
FR-10	UC-02, state diagram	Draft→Pending transition tests
FR-11	UC-03, state diagram	Approve/reject workflow tests
FR-12	UC-03, state diagram	Visibility validation test
FR-13	Check-in flow, ERD(AttendanceCheckIn)	Timestamp persistence tests
NFR-01	Performance requirement	Load testing (500 concurrent)
NFR-03	Security requirement	Credential storage verification
FR-19	Authentication flow + ERD(User.email)	Domain validation tests (reject non-@uvawise.edu)
FR-20	Check-in flow + Class(CheckInService)	NFC/QR/PIN tests + fallback test; verify timestamp stored
FR-21	Role request flow + ERD(RoleRequest)	Submit request test (min 20 chars)
FR-22	Role request admin dashboard + Class(RoleRequestService)	Approve/reject test + reviewer notes saved
FR-23	Account settings + Class(AccountSettingsService)	Update profile/role display/request history tests
NFR-10	Check-in quality requirement	Device-context availability + usability checks
NFR-11	Role workflow security requirement	Privileged endpoint denial tests for non-admin users
NFR-12	Domain enforcement requirement	Domain enforcement tests on login/signup

8 Validation & Reflection

8.1 Validation Summary

This SRS was reviewed for clarity, measurability, and testability. Requirements use consistent terminology, avoid design bias, and include validation methods via the traceability matrix.

8.2 Reflection Questions

- **Hardest requirement to define:** Event visibility constraints (only Approved events appear) because it depends on precise lifecycle/state logic.
- **Did modeling reveal missing requirements?** Yes. Modeling revealed the need for a Draft state and explicit transitions before submission/approval, and use-case exception flows revealed the need for fallback paths when primary check-in fails.
- **Which nonfunctional requirement most influenced system structure?** Maintainability

(separating business logic from presentation) influenced the architecture and class responsibilities.

- **If this project were real, what requirement risks concern you most?** Requirement volatility, especially changes to cultural credit policies or authentication/compliance rules that could force redesign.

8.3 Known Issue / Future Enhancement

Administrator Bootstrapping Problem (critical): If all new accounts default to Student, the system must define how the first Administrator account is created (seed account, instructor-created account, or one-time bootstrap token).